Limen

Inbound API

Read gates, people and events, create visitors and badges, and open a gate from your own software.

VERSION	BASE URL	FORMAT
v1	https://SERVER:8443/api/v1	JSON · HTTPS

00 Overview

The API is **inbound only**: your program calls Limen, Limen never calls your systems.

01 Accounts — who can use the API

02 Authentication — login and token

03 Errors — status codes

04 Endpoints — gates, people, events, visitors, badges, open

05 Typical flow — a complete example

> **Replace** `SERVER` in every example with the address of your Limen server (shown in your browser, e.g. `192.168.1.16`). The server uses HTTPS on port 8443; with the default self-signed certificate, add `-k` to `curl` or install the certificate on the client.

01 Accounts

API accounts are created by an administrator in **System properties → API**. Each account has a username and a password. The password is shown **once** at creation and is stored only as a bcrypt hash.

Full access

Every account can use the whole API. There are no per-account permissions to manage.

One per program

Give each program its own account, so you can revoke one without touching the others.

Revocable

An administrator can revoke an account, issue a new password or delete it. Tokens stop working immediately.

02 Authentication

Send the username and password in the JSON body. You get a token valid for **48 hours**; when it expires, log in again.

REQUEST

```
curl -k -X POST https://SERVER:8443/api/v1/auth/token \
  -H "Content-Type: application/json" \
  -d '{"username": "portineria", "password": "YOUR_PASSWORD"}'
```

RESPONSE `200`

```
{ "access_token": "k3J…", "token_type": "bearer",
  "expires_in": 172800, "expires_at": "2026-11-02T10:00:00Z" }
```

Use the token on every other request:

```
Authorization: Bearer <access_token>
```

- Tokens are random 256-bit values; the server stores only their SHA-256 fingerprint.
- After **5 wrong attempts in 15 minutes** (same client address and username) login answers `429` with a `Retry-After` header.
- The server is meant to be reached over **HTTPS** only, so credentials and tokens are never sent in clear text.

03 Errors

Errors have the form `{"detail": "message"}`.

STATUS	MEANING
`401`	wrong credentials, or missing / invalid / expired token
`404`	object not found
`409`	conflict (badge already assigned, gate disabled)
`422`	invalid body or parameters
`429`	too many failed logins
`503`	gate device offline, command not sent

04 Endpoints

> **Dates.** All date-times are ISO 8601. For **people** they are **local time** (no zone); for **events** they are **UTC** with a trailing `Z`.

GET /gates

List of gates. `mode` is one of `normal`, `scheduled_unlock`, `maintenance`, `emergency`.

RESPONSE `200`

```
[{"id": 1, "name": "Main entrance", "enabled": true, "mode": "normal"}]
```

GET /people

List of people and visitors.

QUERY	DESCRIPTION
`q`	words matched on first and last name
`visitors`	`true` or `false`: only visitors, or only people
`limit`	default 200, max 1000

RESPONSE `200`

```
[{"id": 7, "first_name": "Mario", "last_name": "Rossi", "visitor": true, "active": true,
  "valid_from": null, "valid_until": "2026-11-03T18:00", "badges": ["04A1B2C3"]}]
```

`GET` /events

Latest passages, newest first.

QUERY	DESCRIPTION
`since`	only events from this UTC date-time on
`gate_id`	only this gate
`limit`	default 100, max 1000

Possible `result` values: `granted`, `denied_unknown_credential`, `denied_outside_schedule`, `denied_no_rule`, `denied_credential_disabled`, `denied_credential_expired`, `denied_anti_passback`, `denied_gate_disabled`, `denied_gate_maintenance`, `denied_gate_not_authorized`, `denied_rule_disabled`, `denied_data_outdated`, `door_forced`, `door_left_open`, `remote_open`, `external_open`, `input_activated`, `output_triggered`.

RESPONSE `200`

```
[{"id": 981, "timestamp": "2026-10-30T08:12:44Z", "gate": "Main entrance",
  "result": "granted", "badge": "04A1B2C3", "person_id": 7}]
```

POST /visitors

Creates a visitor. The expiry `valid_until` is **mandatory** and applies to the person and to the badge.

FIELD	DESCRIPTION
`first_name`, `last_name`	required
`valid_until`	required, end of validity (local time)
`valid_from`	optional, start of validity; default: now
`badge_code`	optional, badge to attach right away

```
curl -k -X POST https://SERVER:8443/api/v1/visitors \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"first_name":"Mario","last_name":"Rossi","valid_until":"2026-11-
03T18:00","badge_code":"04A1B2C3"}'
```

Returns `201` with the person object, or `409` if the badge is already assigned.

> A visitor has **no access rules** until you give them some in the web interface (for example by adding them to a group).

POST /people/{id}/badges

Attaches a badge to a person.

```
{"code": "04A1B2C3"}
```

Returns `201` with the person, or `409` if the badge is already assigned.

POST /gates/{id}/open

Sends the open command to the gate. It is fire-and-forget (UDP): `503` if the device is offline.

RESPONSE `200`

```
{"gate_id": 1, "sent": true}
```

05 Typical flow

- `POST /auth/token` once; keep the token and its `expires_at`.
- Call the endpoints with `Authorization: Bearer`.
- On `401`, log in again and retry once.

EXAMPLE IN PYTHON

```
import requests

BASE = "https://SERVER:8443/api/v1"
s = requests.Session(); s.verify = False   # self-signed certificate: better, pass the
certificate path

r = s.post(f"{BASE}/auth/token", json={"username": "portineria", "password": "…"})
token = r.json()["access_token"]
s.headers["Authorization"] = f"Bearer {token}"

v = s.post(f"{BASE}/visitors", json={"first_name": "Mario", "last_name": "Rossi",
"valid_until": "2026-11-03T18:00"}).json()
s.post(f"{BASE}/people/{v['id']}/badges", json={"code": "04A1B2C3"})
```